

UAFS Autonomous Vehicle Challenge 2026



Table of Contents

1. Overview
2. Composite Drawing + Description
3. Claw and Manufacturing process
4. Electrical Schematic + Description
5. Code + Explanation
6. AI Implementation
7. Successes and Failures
8. Build of Materials
9. Appendix (Documentation & Tutorials)

List of Figures

- Fig. 1-Fig. 2: Composite and vehicle drawing
- Fig. 2-Fig. 7: Claw design and manufacturing process
- Fig. 8-Fig. 10: Electrical schematics and description
- Fig. 11-Fig. 12: AI and camera training
- Fig. 13: Test run results
- Fig. 14: Build of materials

Abstract

This project presents the development of an autonomous vehicle for the 2026 AVC competition. The system integrates embedded control, AI-based perception, and mechanical design. A Jetson-based vision system enables object detection, while an Arduino controls actuation. The system demonstrates autonomous navigation and real-time decision making.

1. General Overview

The autonomous vehicle combines multiple engineering disciplines into a single functional system. The goal was to create a platform capable of detecting objects and navigating without human input.

The system uses a Yolo v11 for AI processing and an Arduino Uno for real-time control. Sensors such as an IMU and depth camera provide environmental awareness. The system balances computational performance with real-time responsiveness.

Mechanical Design Overview

The chassis was constructed using a rigid base to support all components. A differential drive system using 12V DC, 300 RPM motors allows for precise movement.

Weight distribution was optimized by placing batteries in the rear. This improves stability and reduces tipping risk. The design prioritizes simplicity and modularity for easy iteration.

Electrical Design Overview

The system is powered by dual VB99 batteries. Key components include the Arduino Uno, Jetson Orin Nano, motor driver, IMU, and cameras. A custom PCB organizes wiring and

improves reliability. Serial communication connects the Jetson and Arduino, allowing AI decisions to control motors.

AI and Vision System Overview

The AI system uses neural networks for object detection. The Intel RealSense D435i provides RGB and depth data.

Training Process (Integrated from Documentation)

The model was trained using the Jetson inference framework: - Dataset created using camera capture - Labels defined in labels.txt - Training performed using SSD model

Example training command:

```
python3 train_ssd.py --dataset-type=voc --
data=data/recMAIN --model-dir=models/test --
batch-size=3 --epochs=90
```

The model was then exported to ONNX format for deployment.

Performance key points

Key metrics considered: - Detection accuracy - Inference speed - Stability in real-time use

2. Composite Drawing + Description

The composite design shows full system integration. Components are placed for optimal performance and accessibility.

The camera is front-mounted for maximum visibility. The IMU is centrally placed for accurate motion tracking. Wiring is minimized through PCB integration.

ARM & CLAW

- Aluminum & carbon fiber arm
- Carbon-fiber epoxy composite claw

5 DOF arm & claw with 1 square meter area, 0.5-meter reach. Capable of 0.2kg load at max span

VEHICLE BODY

- Aluminum lower, acrylic upper
- Water jet fabrication
- 3D printed carbon fiber standoffs, mounts, & casings
- Bent-aluminum bumper

Custom fabricated two-level platform to hold the required hardware and satisfy a maneuverable tri-wheel design.

DRIVING SYSTEM

- Tri-wheeled
- 12V DC, 300 RPM motor
- Two front driver wheels
- One rear caster wheel

Utilizes tri-wheel design. Allows rotation in place, simple hardware, and tight turns.

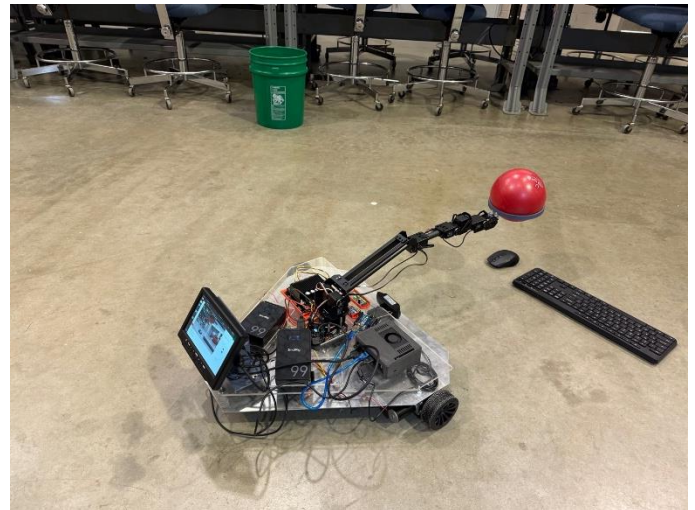
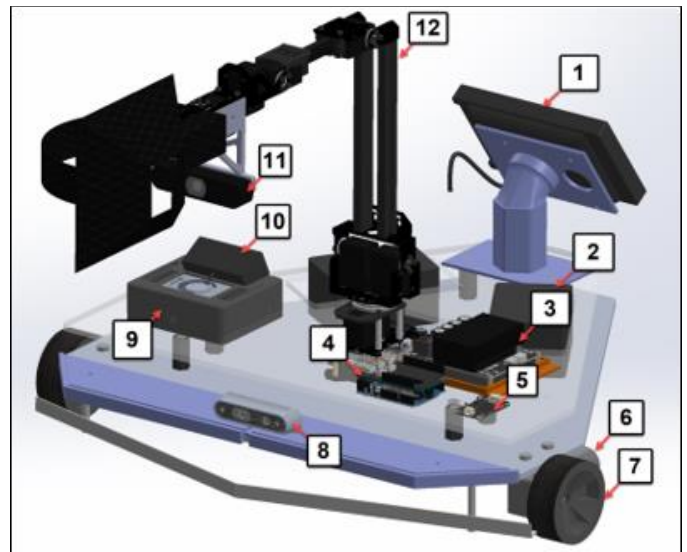


Fig. 1: Vehicle during an Arm demo



3. Claw Design and Manufacturing Process

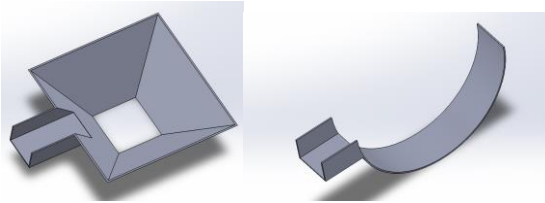


Fig. 3&4: Claw Hand and thumb.

When first designing our claw, we knew that it would need to be a robust but light weight design to not compromise the servo motors on our mechanical arm. The design choice was simple yet reliable. Now we would need to build it to where it would maintain its shape while being light enough for the arm. We decided that it would be best to make it out of carbon fiber.

This is a 3d printed mold based on the solid works drawings of the claws. We chose 3d printing for the mold because it could give us exact dimensions while being cheap and quick. Before applying any carbon fiber, the mold was prepped with release gel and then coated with spray adhesive.

This is the process of laying carbon fiber in the mold, making sure to get it into every groove and corner to reduce the imperfections in the final product. Epoxy is then applied to the fiber and left to cure for a period of 24 hours. Once it is cured, it can be released from the mold, cut to shape, sanded and finished.

This is the final product, accurate, durable, and light weight claws. The perfect fit for our servo-driven robotic arm.

Process is pictured below.

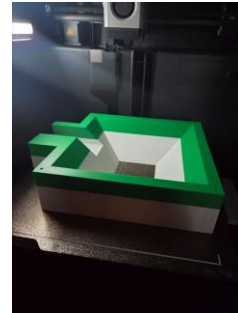


Fig. 5: Cast 3D model

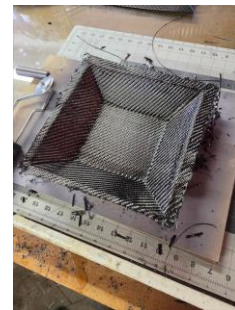


Fig. 6: Carbon fiber application.



Fig. 7: Final claw

4. Electrical Schematic + Description

The schematic defines all system connections. The motor driver interfaces with the Arduino, which processes sensor data.

The Jetson communicates via serial to send high-level commands. Proper grounding and voltage regulation ensure stability.

CUSTOM ARDUINO HAT

- Connects all sensors
- Power Distribution
- Easy Connection

Handles voice commands, sensors, communication with jetson, power distribution

INTEL 435I REALSENSE & EVEET CAM

- Lidar USB Camera
- RGB USB Camera
- Code Support (Libraries)

BATTERY SYSTEM

- Small RigVB99
- Camera Batteries with high amperage outputs and a range of voltages

ROARM M3-Pro

- Lifting Strength
- Smart Servos
- Dedicated Microcontroller Board

JETSON ORIN NANO

- Nvidia Single Board Computer
- Dedicated GPU
- Powerful single board computer with a dedicated GPU which allows for faster AI inferencing and webcam visualizations

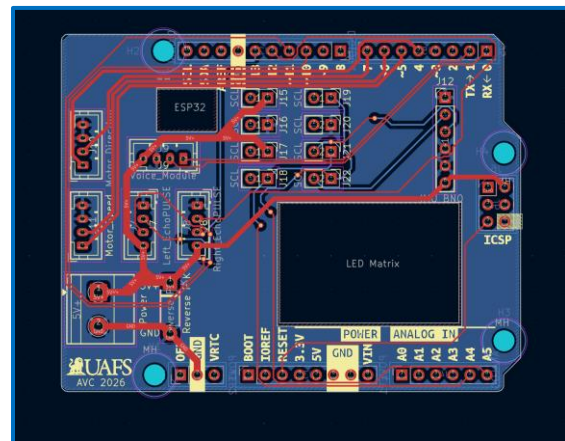


Fig. 8: PCB board "HAT" routing.

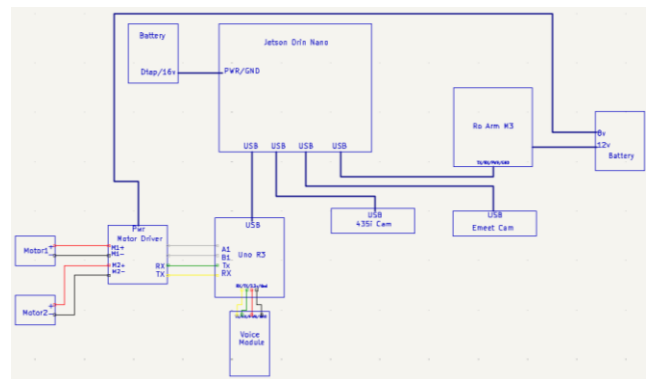


Fig. 9: Simplified Block Diagram.

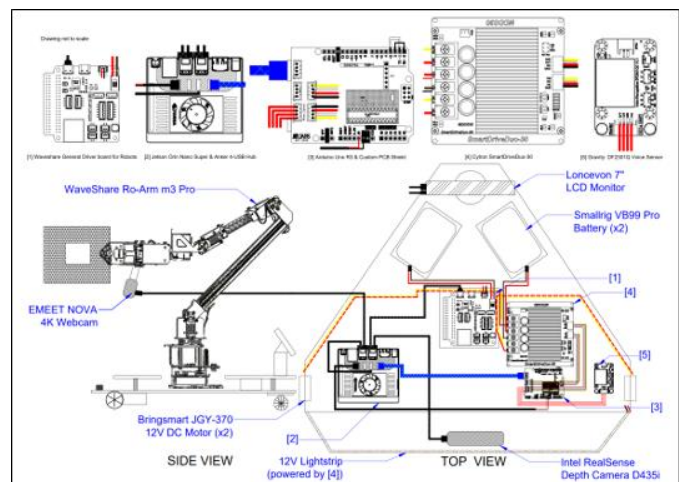


Fig. 10: Electrical overview.

5. Code + Explanation

```
def ik(x, y, z, angles, error):
    error1 = error
    arm1 = 10
    arm2 = 14
    wrist = 7.5

    base_angle_rad = math.atan2(y, x)
    base_angle = math.degrees(base_angle_rad)
    angles[0] = base_angle

    rotated_x = math.hypot(x, y)

    arm1_angle_deg = clamp(angles[1] + 90, -180, 180)
    arm1_rad = math.radians(arm1_angle_deg)
    arm1x = arm1 * math.cos(arm1_rad) * -1
    arm1z = arm1 * math.sin(arm1_rad)

    arm1_angle_world = math.degrees(math.atan2(arm1z, arm1x))

    dx = rotated_x - arm1x
    dz = z - arm1z

    desired_angle_rad = math.atan2(dz, dx)
    desired_angle_deg = math.degrees(desired_angle_rad)

    new_elbow = clamp(arm1_angle_world - desired_angle_deg, -
70, 190)
    angles[2] = new_elbow

    arm2_angle_world = math.radians(arm1_angle_world -
new_elbow)
    finalx = arm1x + arm2 * math.cos(arm2_angle_world)
    finalz = arm1z + arm2 * math.sin(arm2_angle_world)

    error = math.hypot(finalx - rotated_x, finalz - z)
```

Code Highlights and Explanation

- Custom inverse kinematics
- Error protection

The attached code is a snippet from our own inverse kinematics code for the Ro-Arm-M3 pro arm we chose. The company provides some code in the form of a library to handle python communication over serial to the arm but lacked any form of a coordinate system where we could place the end effector at specific coordinates.

Noticing the lack of a coordinate system we wrote our own inverse kinematics code to calculate the angle for each motor to achieve our end position as well as implementing error protection so the arm would not drive itself into the ground or reach outside its own range.

The code on the left utilizes common trigonometry to calculate the angles using tangent, cos, and sin. We treat the complex 3d movements instead as 2 different 2d plane problems that we can solve and then send to the arm.

6.AI Implementation

We utilized YOLOv11 as our base AI model to work with. Utilizing YOLOv11 we redefined our training process that was once very manually intensive to now take advantage of the roboflow training workflow.

Training Workflow

Roboflow allows multiple users to simultaneously annotate AI images. This involved manually boxing the objects we want to train the ai on and labeling them the correct object. Roboflow has many tools such as smart select which speeds up this process allowing much quicker turn around for AI model training. We then are able to use cloud computing to process our ai model into a .pt file we can download. Once we have the file we are able to convert to a .engine file which optimizes the model for the computer we run the conversion on.

AI Results

Our latest AI model in lab conditions was able to achieve the following:

- 60-foot detection for buckets
- 30-foot detection for balls

These results were achieved in lab testing. We can expect about half of these distances successfully in changing environmental conditions

Example detections



Fig. 12: Close range bucket detection example



Fig. 11: Long Range correct AI Detection

7. Successes and Failures

Successes

- Robotic Arm Accuracy
- Multiple systems
- Course Results

Recent Test Run Scores		
Round	Score	Time(m)
1	250	8
2	250	5:50
3	250	6:07
Average:		250/250

Fig. 13: Final test run scores before challenge.

The robotic Arm is our most reliable system. The arm is able to accurately acquire balls over 90 percent of the time with the first pickup attempt. The errors that persist mainly are due to wind, occlusion, and other unrelated factors to the system.

We have successfully integrated multiple systems together effectively and efficiently. These systems include: Camera feed display system, sensor data acquisition from the Arduino uno , AI detection, motor driver control, arm control , manual Bluetooth control with an Xbox controller, and voice activation.

Failures / Challenges

- AI Detection Range
- False detections
- Performance (FPS)
- Power Distribution
- Time constraints

These challenges required iterative debugging and redesign.

AI detection was an issue that persisted throughout our entire training period. Training the AI sufficiently to handle many different light levels, angles, and miscellaneous changes is difficult and requires many images. The balls appear tiny in the video feeds making the detection and our resolution difficult.

The program's performance was a large issue that required many optimizations and then redesigns of entire systems to get a performance level that was satisfactory. Multithreading the program and syncing all systems to run parallel was successful to get the program running over 30 fps per video feed.

Power distribution specifically of various voltages was a big consideration. Our battery system consisting of two smallrigvb99 batteries allows for such voltages as: 5v, 8v, 12v, and 16v. These were necessary considerations for the range of different requirements for the input voltages of the motors, arm, jetson Orin nano, Arduino, sensors, etc.

AVC BOM CUMULATIVE

Supplier (Embedded Link)	Item Number/Name	Description	Quantity per pack	Quantity	Price per Item	Total Price (Before Tax)
Self/Amazon	Vehicle Body	Aluminum board, Plexi glass, screws, etc.	N/A	N/A	N/A	\$150.00
Amazon	Jetson Orin Nano	Single board computer, main computation	1	1	\$250.00	\$250.00
Intel	Intel 435i	Lidar depth and ai camera	1	1	\$334.00	\$334.00
Amazon	Cyton Motor Drive	Motor driver	1	1	\$92.50	\$92.50
Amazon	Eemeet 4k Camera	Arm camera	1	1	\$44.45	\$44.45
Amazon	Wheels		2	1	\$20.00	\$20.00
Amazon	BringSmart Motors		1	2	\$28.97	\$28.97
Amazon	Anker 4 Port Usb Hub	Hub for more ports for cameras	1	1	\$23.99	\$23.99
Amazon	DF Robot Voice Module	module for voice commands	1	1	\$16.90	\$16.90
Amazon	Arduino Uno R3	microcontroller	1	1	\$27.60	\$27.60
Amazon	SmallRig VB99	Main battery's	1	2	\$199.99	\$399.98
Amazon	Display	Small display for debugging	1	1	\$79.99	\$79.99
Waveshare	RoArm M3 Pro	Robotic arm	1	1	\$339.00	\$339.00
Amazon	3d printer filament	3d printing carbon fiber and abs	1	1	\$60.00	\$60.00
					Total Price:	\$1,807.38
					After Tax:	\$1,970.04

Fig. 14: Build of materials total.

8. Appendix

A. Jetson Setup Notes

- Installed JetPack
- Configured Wi-Fi (WPA2 Enterprise)
- Installed dependencies (OpenCV, PyTorch)

B. GitHub Workflow

- `git add .`
- `git commit -m "message"`
- `git push origin main`

C. AI Training Workflow

1. Capture dataset
2. Label data
3. Train model
4. Export ONNX

5. Run detection

D. Useful Commands

- `sudo rm -rf` (delete files)
- `git pull` (update repo)
- python3 scripts for training

E. Motor + Driver Notes

- Encoder wiring used for feedback
- Motor driver logic controls direction and speed

F. Additional Resources

- NVIDIA Jetson documentation
- YOLO / SSD training guides
- OpenCV tutorials